

Práctica 2

Parte 3

En el ámbito de los sistemas empotrados para tiempo real, la gestión eficiente de la concurrencia y la comunicación entre tareas juega un papel crítico. En este contexto, FreeRTOS ofrece una variedad de herramientas para abordar este asunto en hardware con recursos limitados.

La concurrencia en sistemas de tiempo real implica la ejecución simultánea o intercalada de múltiples tareas, cada una con su propio flujo de ejecución y requisitos de recursos. Sin embargo, la gestión inadecuada de la concurrencia puede conducir a problemas como condiciones de carrera, bloqueos, pérdida de datos, etc.

En esta parte de la práctica introducimos colas, grupos de eventos, semáforos y secciones críticas, que permiten a las tareas coordinar sus actividades, compartir recursos y comunicarse de manera segura y eficiente en entornos donde se requiere respuesta en tiempo real.

Sigue los siguientes pasos:

1. En el proyecto STR2024_P2, crea la configuración “parte3_1”. En su fichero main.c copia el siguiente código:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"

#define QUEUE_LENGTH 10
#define ITEM_SIZE sizeof(int)

static const char* TAG = "P2_3_1";

// Tarea del productor
void producer_task(void *pvParameters)
{
    QueueHandle_t xQueue = (QueueHandle_t) pvParameters;
    int i = 0;
    while (1)
    {
        // Genera un nuevo dato
        int item = i++;
        // Envía el dato a la cola
```

```
ESP_LOGI(TAG, "Tarea productor: item %d producido", item);
xQueueSend(xQueue, &item, portMAX_DELAY);

// Simula una pausa entre envíos
vTaskDelay(pdMS_TO_TICKS(500));
}

// Tarea del consumidor
void consumer_task(void *pvParameters)
{
    QueueHandle_t xQueue = (QueueHandle_t) pvParameters;
    while (1)
    {
        // Espera a que haya datos disponibles en la cola
        int item;
        xQueueReceive(xQueue, &item, portMAX_DELAY);

        // Recibe un ítem
        ESP_LOGI(TAG, "Tarea consumidor: item %d consumido", item);

        // Esta espera hace que el consumidor sea muy lento,
        // Para inducir al bloqueo de sincronización
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

void app_main()
{
    // Crea una cola con capacidad para 10 elementos de tipo entero
    QueueHandle_t xQueue;
    xQueue = xQueueCreate(Queue_LENGTH, ITEM_SIZE);

    // Crea las tareas del productor y consumidor
    xTaskCreate(producer_task, "producer_task", 2048, xQueue, 1, NULL);
    xTaskCreate(consumer_task, "consumer_task", 2048, xQueue, 1, NULL);
}
```

Estos métodos bloquearán la tarea si es necesario.
Send bloquea a la espera de una ranura vacía en la cola para poder escribir.
Receive bloquea a la espera de ítem disponible para leer.

Si el valor `portMAX_DELAY` se estableciese en 0, el método sería **no bloqueante**. Es decir, nuestra tarea no se detendría en el caso de no tener espacio para escribir ni nada nuevo que leer. En estos casos, nuestro código se deberá encargar de evaluar el resultado de la función antes de continuar.
<https://www.freertos.org/a00117.html>

Apartado VI: Sincronización mediante colas.

1. Aporta captura de pantalla de la ejecución y responde a las siguientes preguntas:
 - a. Describe qué ocurre al inicio de la ejecución y, después, cuando la cola se llena.
 - b. ¿Qué pasaría si hacemos la cola más grande (incrementamos `Queue_LENGTH`)?
 - c. Explica brevemente para qué puede ser útil utilizar colas bloqueantes.
2. Aporta una modificación en el código que obtenga el mismo resultado pero utilizando `xQueueSend` y `xQueueReceive` no bloqueantes.
 - a. Explica brevemente para qué crees que es de utilidad utilizar colas de forma no bloqueante.

2. Crea la configuración “parte3_2a” del proyecto. En app_main, introduce el siguiente código:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_log.h"

// Macros para las máscaras de bits
#define BIT_0 (1 << 0)
#define BIT_1 (1 << 1)
// ...
// #define BIT_n (1 << n)

static const char* TAG = "P2_3_2a";

// Función que procesa los bits del grupo de eventos
void task_wait_for_event(void *pvParameters)
{
    EventGroupHandle_t event_group = (EventGroupHandle_t)pvParameters;
    EventBits_t bits;
    while (1)
    {
        // Espera a que se establezca el evento BIT_0 o BIT_1
        bits = xEventGroupWaitBits( event_group, BIT_0 | BIT_1,
                                    pdTRUE, pdFALSE, portMAX_DELAY);

        // Como resultado tenemos una palabra de 32 bits disponibles
        if (bits & BIT_0)
        {
            ESP_LOGI(TAG, "BIT_0 detectado");
        }
        if (bits & BIT_1)
        {
            ESP_LOGI(TAG, "BIT_1 detectado");
        }
    }
}

// Función de la tarea que establece el evento BIT_0 periódicamente
void task_set_bit_0(void *pvParameters)
{
    EventGroupHandle_t event_group = (EventGroupHandle_t)pvParameters;
    while (1)
    {
        // Establece el evento BIT_0
        xEventGroupSetBits(event_group, BIT_0);

        // Espera un tiempo antes de establecerlo nuevamente
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}
```

```
// Función de la tarea que establece el evento BIT_1 periódicamente
void task_set_bit_1(void *pvParameters)
{
    EventGroupHandle_t event_group = (EventGroupHandle_t)pvParameters;
    while (1)
    {
        // Establece el evento BIT_1
        xEventGroupSetBits(event_group, BIT_1);

        // Espera un tiempo antes de establecerlo nuevamente
        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

void app_main()
{
    // Crea el grupo de eventos
    EventGroupHandle_t event_group = xEventGroupCreate();

    // Crea las tareas
    xTaskCreate(task_wait_for_event, "task_wait_for_event", 2048, event_group, 1, NULL);
    xTaskCreate(task_set_bit_0, "task_set_bit_0", 2048, event_group, 1, NULL);
    xTaskCreate(task_set_bit_1, "task_set_bit_1", 2048, event_group, 1, NULL);
}
```

Apartado VII: Grupos de eventos

1. Aporta captura de pantalla de la ejecución. Explica brevemente para qué crees que son de utilidad los grupos de eventos.
2. Busca información del método `xEventGroupWaitBits` y responde a las siguientes preguntas:
 - a. ¿Qué significa la expresión `"BIT_0 | BIT_1"`? ¿Qué crees que pasaría si este argumento tuviese el valor `"BIT_0"`? Haz la modificación y lanza el código. Explica brevemente qué ha pasado y trata de justificar por qué.
 - b. ¿Qué ocurre si el timeout (`portMAX_DELAY`) expira sin que ningún bit de los esperados se activa?
 - c. ¿Para qué sirve el tercer argumento (`pdTRUE`)? ¿Qué quiere decir "clear the bits"?
3. Busca información sobre el método `xEventGroupSync` y responde las siguientes cuestiones:
 - a. Explica brevemente para qué crees que es de utilidad.
 - b. Crea la configuración `parte3_2b` en el proyecto y codifica un ejemplo sencillo de uso de este método. Explica brevemente tu código y aporta capturas de pantalla que evidencien su correcto funcionamiento.

3. Crea la configuración "parte3_3a" del proyecto. En `app_main`, introduce el siguiente código:

```
#include <stdio.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/semphr.h>
#include <esp_log.h>
```

```
static const char* TAG = "P2_3_3a";

// Estructura con argumentos para las tareas
typedef struct {
    char*          task_name;
    uint32_t       delay_outside;
    uint32_t       delay_inside;
    SemaphoreHandle_t semaphore;
} Parameters_t;

// Tarea
void task_function(void *parameters)
{
    // Parámetros de configuración
    SemaphoreHandle_t semaphore = ((Parameters_t *)parameters)->semaphore;
    uint32_t delay_inside = ((Parameters_t *)parameters)->delay_inside;
    uint32_t delay_outside = ((Parameters_t *)parameters)->delay_outside;
    char* task_name = ((Parameters_t *)parameters)->task_name;

    while (1)
    {
        // Esperar al semáforo
        if (xSemaphoreTake(semaphore, portMAX_DELAY) == pdTRUE)
        {
            ESP_LOGI(TAG, "%s: Entro de la zona protegida", task_name);

            // Espera dentro de la zona protegida
            vTaskDelay(delay_inside);

            ESP_LOGI(TAG, "%s: Salgo de la zona protegida", task_name);

            // Liberar el semáforo
            xSemaphoreGive(semaphore);
        }
        else
        {
            ESP_LOGI(TAG, "%s: Demasiada espera. Continúo con otra cosa ...", task_name);
        }

        // Espera fuera de la zona protegida
        vTaskDelay(delay_outside);
    }
}

void app_main()
{
    // Crear un semáforo binario
    SemaphoreHandle_t semaphore = xSemaphoreCreateBinary();

    // Libera el semáforo
    xSemaphoreGive(semaphore);
}
```

```
// Parámetros de configuración de las tareas
Parameters_t task1_params = \
{
    .task_name = "Task 1",
    .delay_inside = 500/portTICK_PERIOD_MS,
    .delay_outside = 1000/portTICK_PERIOD_MS,
    .semaphore = semaphore
};
Parameters_t task2_params = \
{
    .task_name = "Task 2",
    .delay_inside = 1000/portTICK_PERIOD_MS,
    .delay_outside = 500/portTICK_PERIOD_MS,
    .semaphore = semaphore
};

// Creación de las tareas
xTaskCreate(task_function, "Task 1", 2048, &task1_params, 1, NULL);
xTaskCreate(task_function, "Task 2", 2048, &task2_params, 1, NULL);
}
```

Apartado VIII: Exclusión mutua

3. Aporta captura de pantalla de la ejecución y responde a las siguientes cuestiones:
 - a. Explica brevemente para qué se está usando el semáforo en el código.
 - b. Utiliza la navegación de declaraciones de VSCode para consultar la definición de SemaphoreHandle_t (control + click). ¿Con qué estructura se implementan internamente los semáforos?
 - c. Busca documentación sobre los diferentes tipos de semáforos que implementa esp-idf. Describe brevemente cada uno de ellos.
 - d. Realiza una ejecución del código cambiando xSemaphoreCreateBinary por xSemaphoreCreateMutex. ¿Funciona igual? En caso afirmativo: ¿cuál crees que es la opción más correcta? Justifica tu respuesta.

4. Crea la configuración “parte3_3b” del proyecto. En app_main, introduce el siguiente código:

```
#include <stdio.h>
#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/semphr.h>
#include <esp_log.h>

static const char* TAG = "P2_3_3b";

// Tarea
void task_function(void *pvParameters)
{
    // Parámetros de configuración
    SemaphoreHandle_t semaphore = (SemaphoreHandle_t) pvParameters;
```

```
while (1)
{
    // Esperar al semáforo
    if (xSemaphoreTake(semaphore, portMAX_DELAY) == pdTRUE)
    {
        ESP_LOGI(TAG, "task_function: Tick");
    }
    else
    {
        ESP_LOGI(TAG, "task_function: Demasiada espera. Continúo con otra cosa ...");
    }
}

}

void app_main()
{
    // Crear un semáforo binario
    SemaphoreHandle_t semaphore = xSemaphoreCreateBinary();

    // Creación de las tareas
    xTaskCreate(task_function, "Task 1", 2048, semaphore, 1, NULL);

    while(1)
    {
        vTaskDelay(1000/portTICK_PERIOD_MS);
        xSemaphoreGive(semaphore);
    }
}
```

Apartado IX: Sincronización mediante semáforos

1. *Aporta captura de pantalla de la ejecución y responde a las siguientes cuestiones:*
 - a. *Explica brevemente para qué se está usando el semáforo en el código.*
 - b. *Realiza una ejecución del código cambiando xSemaphoreCreateBinary por xSemaphoreCreateMutex. ¿Funciona igual? ¿Por qué?*
2. *Crea la configuración parte3_3c y propón un código que utilice la tarea app_main de forma similar al código 3b, pero sincronizando el tick de N instancias de la tarea con semáforos de tipo contador (xSemaphoreCreateCounting(...)). Puedes basarte en la estrategia del código 3a, en la que el puntero *parameters se utiliza para pasar una estructura con el nombre de cada instancia, además del manejador del semáforo.*